Calcul de requêtes acycliques conjonctive dans une base de données

Nathan TILLIER

Les bases de données sont capables de calculer un nombre impressionant de requêtes par secondes. Certaines requêtes sont extrêmement complexes, mais il s'avère qu'une partie d'entre elles appartient à une même catégorie, celle des requêtes acycliques conjonctives.
Ce sujet s'attache à étudier et implémenter entre autres l'algorithme de Yannakakis, permettant de manière assez étonnante de calculer ces requêtes en temps linéaire. Cet algorithme repose sur le fait que leurs variables ne créent pas de cycle dans l'hypergraphe associé, et s'inscrit donc pleinement dans le thème de l'année.

Positionnement thématique

- INFORMATIQUE (Informatique théorique)
- INFORMATIQUE (Informatique pratique)

Mots-clés

Mots-clés (en français)	**Mots-clés** (en anglais)
Algèbre relationnelle	Relational algebra
Base de données	Database
Hypergraphe	Hypergraph
Acyclicité dans les hypergraphes	Acyclicity in hypergraphs

Bibliographie commentée

Un élément crucial d'une infrastructure informatique est la base de donnée sur laquelle des requêtes sont effectuées : celle-ci doit donc répondre à des impératifs, notamment d'efficacité. Certaines requêtes sont extrêmement complexes, mais il s'avère qu'une bonne partie d'entre elles appartient à une même catégorie, celle des requêtes acycliques conjonctives.
Il s'agit par exemple des jointures (`JOIN ON` en SQL), ce qui revient à calculer une partie d'un produit cartésien.

La fondation théorique des bases de données est l'algèbre relationnelle, une théorie introduite par Edgar F. Codd en 1970 [1]. Dans cette dernière, une requête conjonctive est constituée d'un en-tête introduisant les variables utilisées qu'on souhaite récupérer, ainsi que d'une conjonction de conditions portant sur ces variables. On peut prouver que calculer ce type de requête est NP-difficile [2], et on a donc cherché à déterminer un algorithme légèrement plus restrictif déterminant le résultat d'une telle requête en temps polynomial, voire linéaire. Or il s'avère que certaines requêtes conjonctives, dites acycliques, sont calculables en temps linéaire grâce à l'algorithme éponyme de Yannakakis [3]. L'intérêt de cette restriction sur l'acyclicité est notamment qu'elle est peu restrictive dans la pratique tout en restant assez générale.

Cette algorithme repose en partie sur l'introduction d'un hypergraphe associé à une requête. Les hypergraphes sont essentiellement une généralisation des graphes introduite par Claude Berge [4], dans laquelle

une arête peut relier plusieurs sommets.
Ce point de vue différent, liant théorie des hypergraphes et algèbre relationnelle, permet ainsi de construire un arbre nommé arbre de jointure, à partie duquel on peut construire le résultat en trois parcours : deux de précalcul, et un pour récupérer le résultat [5].

Toutefois, à cause de ces trois parcours différents de l'arbre de jointure, la complexité linéaire possède un facteur constant non négligeable en pratique. Une solution possible est par exemple de construire le résultat en deux parcours seulement. De plus, certains documents récents suggèrent certaines modifications à apporter à la construction de l'arbre de jointure, ainsi qu'au parcours de cet arbre, permettant de tirer profit de l'exécution en parallèle [6].

Problématique retenue

Je propose une implémentation d'une base de donnée capable d'exécuter des requêtes acycliques conjonctives en temps linéaire en la taille de l'entrée et de la sortie.

Objectifs du TIPE

Il s'agira dans un premier temps d'étudier les cas d'usage et la formalisation de ce type de requêtes en algèbre relationnelle, puis les algorithmes utilisés pour

- détecter les requêtes appartenant à cette catégorie (réduction Graham-Yu-Ozsoyoglu)
- les exécuter de manière efficace sur une instance (algorithme de Yannakakis)

Ensuite, on implémentera une structure sous-jacente de base de données, puis les algorithmes abordés pour résoudre les requêtes acycliques conjonctives. On discutera des résultats expérimentaux (taille de l'instance, temps de pré-calcul, temps d'exécution total) sur différents exemples.

Enfin, on abordera les modifications possibles de l'algorithme afin de l'optimiser dans la pratique, avant de le comparer à quelques autres algorithmes calculant des requêtes similaires.

Références bibliographiques

[1] Codd E. F. A relational model of data for large shared data banks *Commun. ACM,* 13, (1970), 377-387 DOI: 10.1145/362384.362685.

[2] Koutris P. Foundations of Data Management : Acyclic Conjunctive Queries https://pages.cs.wisc.edu/~paris/cs784-f19/lectures/lecture2.pdf.

[3] Yannakakis M. Algorithms for Acyclic Database Schemes *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7,* (1981).

[4] Berge C. *Graphes et hypergraphes* Dunod (1970).

[5] Hu X. CS848 Fall 2025: Algorithmic Aspects of Query Processing - Traditional Query Processing https://cs.uwaterloo.ca/~xiaohu/courses/CS848/yannakakis-part2.pdf.

[6] Luo Z., Broeck W. V. den, Broeck G. V. den, Wang Y. R. Algorithms for Optimizing Acyclic Queries https://arxiv.org/abs/2509.14144.